



Marie Team - Documentation Technique Laravel

⚙ État	Terminé
🕒 Date de création	@30 janvier 2025 11:53
🕒 Dernière modification	@1 mai 2025 14:55
🏷 Tags	Marie Team Projet
🏷 Compétence	5 - Mettre à disposition des utilisateurs un service informatique
🏷 Sous compétence	Accompagner les utilisateurs dans la mise en place d'un service

SOMMAIRE

[SOMMAIRE](#)

[LE GITHUB DU PROJET](#)

[PARAMETRES DU PROJET](#)

[Configuration BDD](#)

[Les models](#)

[LES COMMANDES DU PROJET](#)

[Créer une commande :](#)

[Génération des trajets :](#)

[Pour toute les liaisons](#)

[Pour une liaison](#)

[Génération des tarifs:](#)

[Pour tout les trajets](#)

[Pour une liaison :](#)

[Génération des capacités:](#)

[Pour tout les bateaux](#)

Pour un bateau :

LE GITHUB DU PROJET

<https://github.com/julien-gournay/marieteam-laravel>

PARAMETRES DU PROJET

Configuration BDD

Dans le fichier `.env` bien vérifier les paramètres de connexion a la base de données.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1 # ou 'localhost'
DB_PORT=3306
DB_DATABASE=marieteam
DB_USERNAME=root
DB_PASSWORD=
```

Les models

Les models se trouvent dans `marieteam > app > Models`

▼ Bateau.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Bateau extends Model
{
    use HasFactory;

    protected $table = 'bateau';
    protected $primaryKey = 'idBateau';

    protected $fillable = ['idCapitaine', 'nomBateau', 'marque'];

    public function capitaine()
    {
        return $this->belongsTo(User::class, 'idCapitaine', 'idPers');
```

```
}  
}
```

▼ Billet.php

```
<?php  
  
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Database\Eloquent\Model;  
  
class Billet extends Model  
{  
    use HasFactory;  
  
    protected $table = 'billet';  
    protected $primaryKey = 'reference';  
    public $incrementing = false; // Clé primaire non incrémentale  
    protected $keyType = 'string';  
  
    protected $fillable = ['reference', 'idType', 'quantite'];  
  
    public function reservation()  
    {  
        return $this->belongsTo(Reservation::class, 'reference', 'reference');  
    }  
  
    public function type()  
    {  
        return $this->belongsTo(Type::class, 'idType', 'idType');  
    }  
}
```

▼ Capacite.php

```
<?php  
  
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Database\Eloquent\Model;  
  
class Capacite extends Model  
{  
    use HasFactory;  
  
    protected $table = 'capacite';
```

```

        protected $fillable = ['idBateau', 'idCategorie', 'capacite'];

        public function bateau()
        {
            return $this->belongsTo(Bateau::class);
        }

        public function categorie()
        {
            return $this->belongsTo(Categorie::class);
        }
    }
}

```

▼ Categorie.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Categorie extends Model
{
    use HasFactory;

    protected $table = 'categorie';
    protected $primaryKey = 'idCategorie';
    public $incrementing = false; // Clé primaire non incrémentale
    protected $keyType = 'string';

    protected $fillable = ['idCategorie', 'libelleCategorie'];
}

```

▼ Client.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Client extends Model
{
    use HasFactory;

    protected $table = 'client';
}

```

```

        protected $primaryKey = 'idClient';

        protected $fillable = ['nom', 'prenom', 'telephone', 'email'];
    }

```

▼ Liaison.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Liaison extends Model
{
    use HasFactory;

    protected $table = 'liaison';
    protected $primaryKey = 'idLiai';
    public $incrementing = false; // Clé primaire non incrémentale
    protected $keyType = 'string';

    protected $fillable = ['idVilleDepart', 'idVilleArrivee', 'duree'];

    public function portDepart()
    {
        return $this->belongsTo(Port::class, 'idVilleDepart', 'idVille');
    }

    public function portArrivee()
    {
        return $this->belongsTo(Port::class, 'idVilleArrivee', 'idVille');
    }
}

```

▼ Periode.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Periode extends Model
{
    use HasFactory;
}

```

```

protected $table = 'periode';
protected $primaryKey = 'idPeriode';

protected $fillable = ['libelle', 'dateDebut', 'dateFin'];
}

```

▼ Port.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Port extends Model
{
    use HasFactory;

    protected $table = 'port';
    protected $primaryKey = 'idVille';
    public $incrementing = false; // Clé primaire non incrémentale
    protected $keyType = 'string';

    protected $fillable = ['nom', 'pays', 'photo', 'description'];
}

```

▼ Reservation.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Reservation extends Model
{
    use HasFactory;

    protected $table = 'reservation';
    protected $primaryKey = 'reference';
    public $incrementing = false; // Clé primaire non incrémentale
    protected $keyType = 'string';

    protected $fillable = ['idClient', 'idTrajet', 'etat'];
}

```

```

public function client()
{
    return $this->belongsTo(Client::class, 'idClient', 'idClient');
}

public function trajet()
{
    return $this->belongsTo(Trajet::class, 'idTrajet', 'idTrajet');
}
}

```

▼ Tarif.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Tarif extends Model
{
    use HasFactory;

    protected $table = 'tarif';
    protected $primaryKey = 'idTarif';

    protected $fillable = ['idLiai', 'idPeriode', 'idType', 'tarif'];

    public function liaison()
    {
        return $this->belongsTo(Liaison::class, 'idLiai', 'idLiai');
    }

    public function periode()
    {
        return $this->belongsTo(Periode::class, 'idPeriode', 'idPeriode');
    }

    public function type()
    {
        return $this->belongsTo(Type::class, 'idType', 'idType');
    }
}

```

▼ Trajet.php

```

<?php

namespace App\Models;

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Trajet extends Model
{
    use HasFactory;

    // Nom de la table dans la base de données
    protected $table = 'trajet';

    // Nom de la clé primaire
    protected $primaryKey = 'idTrajet';

    // Désactiver les timestamps (created_at, updated_at)
    public $timestamps = false;

    // Colonnes remplies
    protected $fillable = [
        'idLiaison',
        'idBateau',
        'dateDepart',
        'heureDepart',
        'dateArrive',
        'heureArrive',
    ];

    /**
     * Relation avec la table Liaison
     */
    public function liaison()
    {
        return $this->belongsTo(Liaison::class, 'idLiaison', 'idLiai');
    }

    /**
     * Relation avec la table Bateau
     */
    public function bateau()
    {
        return $this->belongsTo(Bateau::class, 'idBateau', 'idBateau');
    }
}

```

```
}  
}
```

▼ Type.php

```
<?php  
  
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Database\Eloquent\Model;  
  
class Type extends Model  
{  
    use HasFactory;  
  
    protected $table = 'type';  
    protected $primaryKey = 'idType';  
    public $incrementing = false; // Clé primaire non incrémentale  
    protected $keyType = 'string';  
  
    protected $fillable = ['idType', 'libelleType'];  
}
```

▼ User.php

```
<?php  
  
namespace App\Models;  
  
use Illuminate\Contracts\Auth\MustVerifyEmail;  
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Foundation\Auth\User as Authenticatable;  
use Illuminate\Notifications\Notifiable;  
use Laravel\Sanctum\HasApiTokens;  
  
class User extends Authenticatable  
{  
    use HasApiTokens, HasFactory, Notifiable;  
  
    /**  
     * The attributes that are mass assignable.  
     *  
     * @var array<int, string>  
     */  
    protected $fillable = [  
        'name',  
        'email',
```

```

        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var array<int, string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * The attributes that should be cast.
     *
     * @var array<string, string>
     */
    protected $casts = [
        'email_verified_at' => 'datetime',
    ];
}

```

LES COMMANDES DU PROJET

Les commandes se trouve dans `marieteam > app>Console > Commands`

Créer une commande :

```
php artisan make:commande {nomCommande}
```

Génération des trajets :

Pour toute les liaisons

```
php artisan generate:trajet {dateDebut} {dateFin} {heureDebut min} {heureDebut max} {trajetJourMin} {trajetJourMax}
```

```
php artisan generate:trajets 2025-12-01 2025-12-31 06:00 22:00 5 10
```

Conditions commande :

**Le nombre de trajets par jour doit être au minimum de 5 et maximum 10 trajets (pouvant être modifié dans le code) .*

******L'heure de fin est calculé selon la durée de la liaison.

```

$signature = 'generate:trajets
              {dateDebut : Date de début au format YYYY-MM-DD}
              {dateFin : Date de fin au format YYYY-MM-DD}

```

```
{plageDebut : Heure de début de la plage horaire (format HH:mm)}  
{plageFin : Heure de fin de la plage horaire (format HH:mm)}  
{trajetsParJourMin=5 : Nombre minimum de trajets par jour (minimum 5)}  
{trajetsParJourMax=10 : Nombre maximum de trajets par jour (maximum 10)}';
```



Par défaut pour Marie Team:

Date début : 1 janvier 2025

Date fin : 31 janvier 2025

Heure premier départ : 06h00

Heure dernier départ : 22h00

Nombre de trajets/Jour : Entre 5 et 10

▼ Le code complet `GenerateTrajets.php`

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use App\Models\Liaison;
use App\Models>Trajet;
use App\Models>Bateau;
use Carbon\Carbon;

class GenerateTrajets extends Command
{
    /**
     * Nom et description de la commande.
     */
    protected $signature = 'generate:trajets
                        {dateDebut : Date de début au format YYYY-MM-DD}
                        {dateFin : Date de fin au format YYYY-MM-DD}
                        {plageDebut : Heure de début de la plage horaire (format HH:mm)}
                        {plageFin : Heure de fin de la plage horaire (format HH:mm)}
                        {trajetsParJourMin=5 : Nombre minimum de trajets par jour (minimum 5)}
                        {trajetsParJourMax=10 : Nombre maximum de trajets par jour (maximum 10)}';

    protected $description = 'Génère des trajets automatiquement selon les paramètres.';

    public function handle()
    {
        // Récupération des paramètres
        $dateDebut = $this->argument('dateDebut');
        $dateFin = $this->argument('dateFin');
        $plageDebut = $this->argument('plageDebut');
        $plageFin = $this->argument('plageFin');
        $trajetsParJourMin = (int) $this->argument('trajetsParJourMin');
        $trajetsParJourMax = (int) $this->argument('trajetsParJourMax');

        // Validation des paramètres
        if ($trajetsParJourMin < 5 || $trajetsParJourMax > 10 || $trajetsParJourMin > $trajetsParJourMax) {
            $this->error('Le nombre de trajets par jour doit être compris entre 5 et 10, et trajetsParJourMin doit être inférieur à trajetsParJourMax');
            return;
        }

        $plageDebutTime = Carbon::createFromTimeString($plageDebut);
        $plageFinTime = Carbon::createFromTimeString($plageFin);

        if ($plageDebutTime->greaterThanOrEqualTo($plageFinTime)) {
            $this->error('La plage de début doit être antérieure à la plage de fin');
            return;
        }

        // Génération des trajets
        $liaisons = Liaison::all();
        foreach ($liaisons as $liaison) {
            $trajets = [];
            $date = $dateDebut;
            while ($date <= $dateFin) {
                $trajet = new Trajet();
                $trajet->liaison_id = $liaison->id;
                $trajet->date = $date;
                $trajet->plage_debut = $plageDebut;
                $trajet->plage_fin = $plageFin;
                $trajet->save();
                $trajets[] = $trajet;
                $date = $date->addDay();
            }
        }
    }
}
```

```

        $this->error('La plage horaire de début doit être avant la plage horaire de fin.');
```

```

        return;
    }

    $liaisons = Liaison::all();
    $bateaux = Bateau::all();

    foreach ($liaisons as $liaison) {
        $this->info("Génération des trajets pour la liaison : {$liaison->idLiai}");

        $dureeLiaisonMinutes = $this->convertTimeToMinutes($liaison->duree);

        $currentDate = Carbon::createFromFormat('Y-m-d', $dateDebut);

        while ($currentDate->format('Y-m-d') <= $dateFin) {
            // Nombre de trajets à générer pour ce jour
            $trajetsParJour = rand($trajetsParJourMin, $trajetsParJourMax);

            $this->info("Date : {$currentDate->format('Y-m-d')} ($trajetsParJour)");

            for ($i = 0; $i < $trajetsParJour; $i++) {
                $bateau = $bateaux->random();

                $heureDepart = $this->generateRandomTime($plageDebutTime, $plageFinTime);
                $heureArrivee = (clone $heureDepart)->addMinutes($dureeLiaisonMinutes);

                $dateArrivee = $currentDate;

                if ($heureArrivee->isBefore($heureDepart)) {
                    $dateArrivee = $currentDate->copy()->addDay();
                }

                Trajet::create([
                    'idLiaison' => $liaison->idLiai,
                    'idBateau' => $bateau->idBateau,
                    'dateDepart' => $currentDate->format('Y-m-d'),
                    'heureDepart' => $heureDepart->format('H:i:s'),
                    'dateArrivee' => $dateArrivee->format('Y-m-d'),
                    'heureArrivee' => $heureArrivee->format('H:i:s'),
                ]);
            }

            $currentDate->addDay();
        }
    }

    $this->info('Génération des trajets terminée.');
```

```

/**
 * Génère une heure aléatoire entre deux plages horaires.
 *
 * @param Carbon $start
 * @param Carbon $end
 * @return Carbon
 */
private function generateRandomTime(Carbon $start, Carbon $end)
{
    $diffInMinutes = $start->diffInMinutes($end);
    $randomMinutes = rand(0, $diffInMinutes);

    return $start->copy()->addMinutes($randomMinutes);
}

/**
 * Convertit une durée au format HH:MM:SS en minutes.
 *
 * @param string $time
 * @return int
 */
private function convertTimeToMinutes($time)
{
    list($hours, $minutes, $seconds) = explode(':', $time);
    return ($hours * 60) + $minutes + ($seconds / 60);
}
}

```

Pour une liaison

```
php artisan generate:trajet {idLiaison} {dateDebut} {dateFin} {heureDebut min} {heureDebut max} {trajetJourMin} {trajetJourMax}
```

```
php artisan generate:untrajet CAL-DOU 2025-02-10 2025-02-10 08:00 18:00 5 6
```

Génération des trajets pour la liaison : CAL-DOU

Date : 2025-02-10 (5 trajets)

Génération des trajets terminée.

▼ Le code complet `GenerateUnTrajet.php`

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use App\Models\Liaison;
use App\Models\Trajet;
use App\Models\Bateau;
use Carbon\Carbon;

class GenerateUnTrajet extends Command
{
    protected $signature = 'generate:untrajet
        {idLiaison : ID de la liaison}
        {dateDebut : Date de début au format YYYY-MM-DD}
        {dateFin : Date de fin au format YYYY-MM-DD}
        {plageDebut : Heure de début de la plage horaire (format HH:mm)}
        {plageFin : Heure de fin de la plage horaire (format HH:mm)}
        {trajetsParJourMin=5 : Nombre minimum de trajets par jour (minimum 5)}
        {trajetsParJourMax=10 : Nombre maximum de trajets par jour (maximum 10) }';

    protected $description = 'Génère des trajets pour une liaison spécifique selon les paramètres';

    public function handle()
    {
        $idLiaison = $this->argument('idLiaison');
        $dateDebut = $this->argument('dateDebut');
        $dateFin = $this->argument('dateFin');
        $plageDebut = $this->argument('plageDebut');
        $plageFin = $this->argument('plageFin');
        $trajetsParJourMin = (int) $this->argument('trajetsParJourMin');
        $trajetsParJourMax = (int) $this->argument('trajetsParJourMax');

        if ($trajetsParJourMin < 5 || $trajetsParJourMax > 10 || $trajetsParJourMin > $trajetsParJourMax) {
            $this->error('Le nombre de trajets par jour doit être compris entre 5 et 10, et trajetsParJourMin doit être inférieur à trajetsParJourMax');
            return;
        }

        $plageDebutTime = Carbon::createFromTimeString($plageDebut);
        $plageFinTime = Carbon::createFromTimeString($plageFin);

        if ($plageDebutTime->greaterThanOrEqualTo($plageFinTime)) {
            $this->error('La plage horaire de début doit être avant la plage horaire de fin.');
```

```

    $liaison = Liaison::find($idLiaison);

    if (!$liaison) {
        $this->error("Aucune liaison trouvée avec l'ID : {$idLiaison}");
        return;
    }

    $this->info("Génération des trajets pour la liaison : {$liaison->idLiaison}");

    $dureeLiaisonMinutes = $this->convertTimeToMinutes($liaison->duree);
    $bateaux = Bateau::all();
    $currentDate = Carbon::createFromFormat('Y-m-d', $dateDebut);

    while ($currentDate->format('Y-m-d') <= $dateFin) {
        $trajetsParJour = rand($trajetsParJourMin, $trajetsParJourMax);
        $this->info("Date : {$currentDate->format('Y-m-d')} ($trajetsParJour trajets)");

        for ($i = 0; $i < $trajetsParJour; $i++) {
            $bateau = $bateaux->random();
            $heureDepart = $this->generateRandomTime($plageDebutTime, $plageFinTime);
            $heureArrivee = (clone $heureDepart)->addMinutes($dureeLiaisonMinutes);
            $dateArrivee = $currentDate;

            if ($heureArrivee->isBefore($heureDepart)) {
                $dateArrivee = $currentDate->copy()->addDay();
            }

            Trajet::create([
                'idLiaison' => $liaison->idLiaison,
                'idBateau' => $bateau->idBateau,
                'dateDepart' => $currentDate->format('Y-m-d'),
                'heureDepart' => $heureDepart->format('H:i:s'),
                'dateArrivee' => $dateArrivee->format('Y-m-d'),
                'heureArrivee' => $heureArrivee->format('H:i:s'),
            ]);
        }

        $currentDate->addDay();
    }

    $this->info('Génération des trajets terminée.');
```

```

private function generateRandomTime(Carbon $start, Carbon $end)
{
    $diffInMinutes = $start->diffInMinutes($end);
    $randomMinutes = rand(0, $diffInMinutes);

```

```

        return $start->copy()->addMinutes($randomMinutes);
    }

    private function convertTimeToMinutes($time)
    {
        list($hours, $minutes, $seconds) = explode(':', $time);
        return ($hours * 60) + $minutes + ($seconds / 60);
    }
}

```

Génération des tarifs:

Pour tout les trajets

php artisan generate:tarifs

```

php artisan generate:tarifs
Liaison: ALC-CIU (ALC → CIU)
Quel est le prix de base pour le type A1 pour la liaison ALC-CIU ? [10]:
> 95
Tarif généré : Liaison ALC-CIU, Type A1, Prix 95
Tarif généré : Liaison ALC-CIU, Type A2, Prix 85.5
Tarif généré : Liaison ALC-CIU, Type A3, Prix 71.25
Tarif généré : Liaison ALC-CIU, Type B1, Prix 5
Tarif généré : Liaison ALC-CIU, Type B2, Prix 123.5
Tarif généré : Liaison ALC-CIU, Type C1, Prix 145.35
Tarif généré : Liaison ALC-CIU, Type C2, Prix 171
Tarif généré : Liaison ALC-CIU, Type C3, Prix 175.75
Tarif généré : Liaison ALC-CIU, Type D1, Prix 204.25
Tarif généré : Liaison ALC-CIU, Type D2, Prix 199.5

```

Conditions :

**Les prix sont générés selon le pourcentage du prix de base donné.*

```

$types = [
    'A1' => $basePrice,
    'A2' => $basePrice * 0.90,
    'A3' => $basePrice * 0.75,
    'B1' => 5,
    'B2' => $basePrice * 1.30,
    'C1' => $basePrice * 1.53,
    'C2' => $basePrice * 1.80,
    'C3' => $basePrice * 1.85,
    'D1' => $basePrice * 2.15,
    'D2' => $basePrice * 2.10,

```

▼ Le code complet `GenerateTarifs.php`

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\DB;

class GenerateTarifs extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'generate:tarifs';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = 'Génère les tarifs pour chaque liaison et chaque type';

    /**
     * Execute the console command.
     */
    public function handle()
    {
        // Récupérer toutes les liaisons depuis la base de données
        $liaisons = DB::table('liaison')->get();

        if ($liaisons->isEmpty()) {
            $this->error('Aucune liaison trouvée dans la base de données.');
```

return Command::FAILURE;

}

// Boucle sur chaque liaison

foreach (\$liaisons as \$liaison) {

 \$this->info("Liaison: {"\$liaison->idLiai"} {"\$liaison->idvilleDepart"} → {"\$liaison->idville

 // Demander le prix de base pour le type A1

 \$basePrice = \$this->ask("Quel est le prix de base pour le type A1 pour la liaison {"\$

 // Assurer que le prix est un nombre valide

 if (!is_numeric(\$basePrice) || \$basePrice <= 0) {

```

        $this->error("Prix de base invalide pour la liaison {$liaison->idLiai}. Ignorée.");
        continue;
    }

    $basePrice = (float) $basePrice;

    // Définir les ajustements pour chaque type
    $types = [
        'A1' => $basePrice,
        'A2' => $basePrice * 0.90,
        'A3' => $basePrice * 0.75,
        'B1' => 5,
        'B2' => $basePrice * 1.30,
        'C1' => $basePrice * 1.53,
        'C2' => $basePrice * 1.80,
        'C3' => $basePrice * 1.85,
        'D1' => $basePrice * 2.15,
        'D2' => $basePrice * 2.10,
    ];

    // Insérer chaque type et tarif dans la table tarifs
    foreach ($types as $type => $price) {
        DB::table('tarif')->insert([
            'idLiaison' => $liaison->idLiai,
            'idPeriode' => 2, // Période fixe
            'idType' => $type,
            'tarif' => round($price, 2), // Prix arrondi à 2 décimales
        ]);
        $this->info("Tarif généré : Liaison {$liaison->idLiai}, Type {$type}, Prix {$price}")
    }
}

$this->info('Génération des tarifs terminée avec succès.');
```

```
return Command::SUCCESS;
```

Pour une liaison :

```
php artisan generate:untarif {idLiaison} {idPeriode}
```

Creation des tarifs pour la liaison Douvre - Calais pendant la periode 2.

```
php artisan generate:untarif DOU-CAL 2
```

Liaison: DOU-CAL (DOU → CAL)

Quel est le prix de base pour le type A1 pour la liaison DOU-CAL ? [10]:
> #Saisir le prix de base

▼ Le code complet `GenerateUnTarif.php`

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\DB;

class GenerateUnTarif extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'generate:untarif {idLiaison} {idPeriode}';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = 'Génère les tarifs pour une liaison et une période spécifique';

    /**
     * Execute the console command.
     */
    public function handle()
    {
        $idLiaison = $this->argument('idLiaison');
        $idPeriode = $this->argument('idPeriode');

        // Vérifier si la liaison existe
        $liaison = DB::table('liaison')->where('idLiai', $idLiaison)->first();
        if (!$liaison) {
            $this->error("Aucune liaison trouvée avec l'ID $idLiaison.");
            return Command::FAILURE;
        }

        $this->info("Liaison: {$liaison->idLiai} ({$liaison->idvilleDepart} → {$liaison->idvilleArrivee})");

        // Demander le prix de base pour le type A1
        $basePrice = $this->ask("Quel est le prix de base pour le type A1 pour la liaison {$liaison->idLiai} ?");

        if (!is_numeric($basePrice) || $basePrice <= 0) {
            $this->error("Prix de base invalide. Opération annulée.");
        }
    }
}
```

```

        return Command::FAILURE;
    }

    $basePrice = (float) $basePrice;

    // Définir les ajustements pour chaque type
    $types = [
        'A1' => $basePrice,
        'A2' => $basePrice * 0.90,
        'A3' => $basePrice * 0.75,
        'B1' => 5,
        'B2' => $basePrice * 1.30,
        'C1' => $basePrice * 1.53,
        'C2' => $basePrice * 1.80,
        'C3' => $basePrice * 1.85,
        'D1' => $basePrice * 2.15,
        'D2' => $basePrice * 2.10,
    ];

    // Insérer chaque type et tarif dans la table tarifs
    foreach ($types as $type => $price) {
        DB::table('tarif')->insert([
            'idLiaison' => $idLiaison,
            'idPeriode' => $idPeriode,
            'idType' => $type,
            'tarif' => round($price, 2),
        ]);
        $this->info("Tarif généré : Liaison $idLiaison, Type $type, Prix $price");
    }

    $this->info('Génération des tarifs terminée avec succès.');
```

```

    return Command::SUCCESS;
}
}

```

Génération des capacités:

Pour tout les bateaux

```
php artisan generate:capacities
```

```

php artisan generate:capacities
Bateau : 1 (MS Stena Britannica)
Catégorie : A, Capacité : 110
Catégorie : B, Capacité : 50
Catégorie : C, Capacité : 110
Catégorie : D, Capacité : 10

```

Bateau : 2 (DAEWOO SHIPBUILDING)

...

Conditions :

```
$categories = [  
'A' ⇒ [80, 500], // Catégorie A : Entre 80 et 500, arrondi à la dizaine près  
'B' ⇒ [10, 100], // Catégorie B : Entre 10 et 100  
'C' ⇒ [20, 150], // Catégorie C : Entre 20 et 150  
'D' ⇒ [5, 50], // Catégorie D : Entre 5 et 50
```

▼ Le code complet `GenerateCapacites.php`

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Artisan;

class GenerateCapacites extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'generate:capacites';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = 'Génère les capacités pour chaque bateau et chaque catégorie';

    /**
     * Execute the console command.
     */
    public function handle()
    {
        // Récupérer tous les bateaux
        $bateaux = DB::table('bateau')->get();

        if ($bateaux->isEmpty()) {
            $this->error("Aucun bateau trouvé dans la base de données.");
            return Command::FAILURE;
        }

        // Catégories avec leurs intervalles de valeurs
        $categories = [
            'A' => [80, 500], // Catégorie A : Entre 80 et 500, arrondi à la dizaine près
            'B' => [10, 100], // Catégorie B : Entre 10 et 100
            'C' => [20, 150], // Catégorie C : Entre 20 et 150
            'D' => [5, 50],   // Catégorie D : Entre 5 et 50
        ];
    }
}
```

```

// Boucle pour chaque bateau
foreach ($bateaux as $bateau) {
    $this->info("Bateau : {$bateau->idBateau} ({$bateau->nomBateau})");

    // Boucle pour chaque catégorie
    foreach ($categories as $categorie => $interval) {
        $capacite = rand($interval[0], $interval[1]); // Génération aléatoire dans l'intervalle
        $capacite = round($capacite / 10) * 10; // Arrondi à la dizaine près

        // Insérer ou mettre à jour la capacité dans la table
        DB::table('capacite')->updateOrCreate(
            [
                'idBateau' => $bateau->idBateau,
                'idCategorie' => $categorie,
            ],
            [
                'capacite' => $capacite,
            ]
        );

        $this->info("  Catégorie : {$categorie}, Capacité : {$capacite}");
    }
}

$this->info("Génération des capacités terminée avec succès.");
return Command::SUCCESS;
}
}

```

Pour un bateau :

```
php artisan generate:unecapacite {idBateau}
```

```

php artisan generate:unecapacite 3
Bateau : 3 (MIPO DOCKYARD)
Catégorie : A, Capacité : 100
Catégorie : B, Capacité : 40
Catégorie : C, Capacité : 60
Catégorie : D, Capacité : 10
Génération des capacités terminée avec succès pour le bateau ID 3.

```

▼ Le code complet [GenerateUneCapacite.php](#)

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\DB;

class GenerateUneCapacite extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'generate:unecapacite {idBateau}';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = "Génère les capacités pour un bateau donné par son ID";

    /**
     * Execute the console command.
     */
    public function handle()
    {
        $idBateau = $this->argument('idBateau');

        // Vérifier si le bateau existe
        $bateau = DB::table('bateau')->where('idBateau', $idBateau)->first();

        if (!$bateau) {
            $this->error("Aucun bateau trouvé avec l'ID {$idBateau}.");
            return Command::FAILURE;
        }

        $this->info("Bateau : {$bateau->idBateau} ({$bateau->nomBateau})");

        // Catégories avec leurs intervalles de valeurs
        $categories = [
            'A' => [80, 500], // Catégorie A : Entre 80 et 500, arrondi à la dizaine près
            'B' => [10, 100], // Catégorie B : Entre 10 et 100
            'C' => [20, 150], // Catégorie C : Entre 20 et 150
        ];
    }
}
```

```

        'D' => [5, 50],    // Catégorie D : Entre 5 et 50
    ];

    // Boucle pour chaque catégorie
    foreach ($categories as $categorie => $interval) {
        $capacite = rand($interval[0], $interval[1]); // Génération aléatoire dans l'intervalle
        $capacite = round($capacite / 10) * 10;      // Arrondi à la dizaine près

        // Insérer ou mettre à jour la capacité dans la table
        DB::table('capacite')->updateOrCreate(
            [
                'idBateau' => $idBateau,
                'idCategorie' => $categorie,
            ],
            [
                'capacite' => $capacite,
            ]
        );

        $this->info(" Catégorie : {$categorie}, Capacité : {$capacite}");
    }

    $this->info("Génération des capacités terminée avec succès pour le bateau ID {$idBateau}");
    return Command::SUCCESS;
}
}

```